# Exercise 3 — Algorithm Deconstruction Challenge

## Exercise Description

Select one of the practical algorithms (task priority sorting, task text parser, or task-list merging), use the "Deciphering Complex Functions and Algorithms" prompts to break it down, document your understanding (with diagrams), capture insights, and answer the reflection questions.

**Algorithm chosen:** Task Priority Sorting (`task_priority.py`, Python).

---

## 1. What the algorithm does

Three cooperating functions turn an unordered list of tasks into a ranked "what should I do next" list:

- `calculate_task_score(task)` → an integer importance score (pure function of the task's fields).
- `sort_tasks_by_importance(tasks)` → tasks ordered by score, highest first.
- `get_top_priority_tasks(tasks, limit=5)` → the top N of that ordering.

## 2. Step-by-step breakdown of `calculate_task_score`

| Step | Rule | Contribution |
|---|---|---|
| Base | `priority_weight × 10`, weights LOW 1 / MEDIUM 2 / HIGH 4 / URGENT 6 | 10, 20, 40, or 60 |
| Due date | overdue `+35`; today `+20`; ≤2 days `+15`; ≤7 days `+10`; else `0` | 0–35 |
| Status | DONE `−50`; REVIEW `−15`; else `0` | −50 to 0 |
| Tags | any of blocker/critical/urgent `+8` | 0 or 8 |
| Freshness | updated < 1 day ago `+5` | 0 or 5 |

The contributions are **additive and independent**; the final score is their sum. Note the priority weights are **non-linear** — the HIGH→URGENT jump (4→6) is larger than the lower steps, so URGENT is emphasised.

### Worked example

`HIGH` priority, due in 1 day, status `TODO`, tag `critical`, updated today: `40 (base) + 15 (due ≤2d) + 0 (status) + 8 (tag) + 5 (fresh) = 68`.

A `MEDIUM` task that is overdue: `20 + 35 = 55` — which **outranks** a `HIGH` task with no due date (`40`). Key insight: **due-date urgency can dominate raw priority.**

## 3. Sorting & top-N

```
sort_tasks_by_importance(tasks):
    task_scores = [(calculate_task_score(t), t) for t in tasks]     # score once per task
    return [t for _, t in sorted(task_scores, key=lambda x: x[0], reverse=True)]
```

- Scores are computed **once per task** (not on every comparison) — good, avoids O(n log n) score recomputation.
- The `key=lambda x: x[0]` is essential: it sorts on the score only, so Python never tries to compare two `Task` objects when scores tie (which would raise `TypeError`).
- `get_top_priority_tasks` simply slices `[:limit]`.

**Complexity:** scoring is O(1) per task → O(n) to score all; sorting is O(n log n); overall **O(n log n)**, O(n) extra space for the tuple list.

## 4. Diagram

```
tasks ──► [ calculate_task_score(t) for each t ]  (O(n))
             │  base(priority×10) + due_bonus + status_penalty + tag_boost + freshness
             ▼
        [(score, task), ...] ──► sorted(desc by score) (O(n log n)) ──► [task, task, ...]
                                                                          └► [:limit] top-
N
```

## 5. Edge cases & risks

- **Tie-breaking is unspecified** — equal scores keep input order (Python's sort is stable), so ordering among ties is arbitrary from the user's view.
- **`days_until_due` uses `.days`** on a `timedelta`, which truncates toward zero; a task due in 47 hours reports `1` day. Boundary behaviour around "today/overdue" can be surprising near midnight.
- **No caching across calls** — fine here, but scores are recomputed every sort.
- **Magic numbers** — all weights are hard-coded; extracting them to a config dict would make tuning and testing easier.

---

## Reflection Questions

1. **How did the AI explanation change your understanding?** It made the priority/due-date *interaction* explicit — I'd assumed priority dominated, but tracing concrete tasks showed urgency frequently wins.

2. **What was still difficult after the explanation?** The `.days` truncation semantics around day boundaries — worth a targeted unit test rather than reasoning alone.
3. **How would you explain it to another junior developer?** "Each task earns points for being important (priority), being due soon, being fresh, and being tagged critical — and loses points for being done or in review. We add the points up and sort highest-first."
4. **Did you test this understanding against AI?** Yes — I posed the "overdue MEDIUM vs. HIGH-no-due" case and confirmed 55 > 40.
5. **How might you improve the algorithm?** Extract weights to a configurable table; add explicit tie-breakers (e.g. earlier due date, then created_at); document the `.days` truncation or switch to a rounded/total-seconds comparison; add unit tests for each scoring component.

---

## Submission for Exercise 3 — Algorithm Deconstruction Challenge

**1. Algorithm selected:** Task Priority Sorting (`calculate_task_score` / `sort_tasks_by_importance` / `get_top_priority_tasks`), Python.

**2. Documented understanding (with diagram):** additive scoring table (base priority ×10 with non-linear weights, due-date urgency, status penalties, tag boost, freshness), the score-once-then-sort flow, and the O(n log n) / O(n) complexity — see the diagram above.

**3. Insights & learning points:** due-date urgency can outrank raw priority; the `key=lambda x: x[0]` prevents `Task`-object comparison on ties; weights are non-linear to emphasise URGENT; `.days` truncation is a hidden boundary risk.

**4. Reflection answers:** provided above — the biggest shift was seeing the priority/urgency interaction; the main residual uncertainty is day-boundary truncation; improvements are configurable weights, explicit tie-breaking, and per-component tests.